Kathmandu University
Department of Computer Science and Engineering
Dhulikhel, Kavre

A Project Report
on
"Election Management System"

[Code No.: COMP 202]

(For partial fulfillment of Year II / Semester I in Computer Engineering)

Submitted by
Prabin Babu Basel (Regd. No. 037958-24)

Submitted to
Mr. Sagar Acharaya
Department of Computer Science and Engineering

February 23, 2026

Acknowledgements

I would like to express my sincere gratitude to everyone who contributed to the successful completion of this project, “Election Management System.”

First and foremost, I would like to extend deepest appreciation to Mr. Sagar Acharya, Lecturer in the Department of Computer Science and Engineering, Kathmandu University, for his continuous guidance and support throughout the project. His expertise in data structures, algorithm design, and C programming, along with his constructive feedback and problem-solving approach, were invaluable at every stage of development. His encouragement greatly enhanced the quality and direction of this work.

I am also grateful to the Department of Computer Science and Engineering, Kathmandu University, for providing the necessary academic resources, computing facilities, and a positive learning environment that made the successful completion of this project possible.

I would like to express my appreciation to my friends for their ideas, motivation, and support throughout the project.

Finally, I sincerely thank my family for their constant encouragement, patience, and understanding, which played a crucial role in helping me complete this project successfully.

Abstract

Efficient and transparent management of election data poses a significant computational challenge, particularly when handling large numbers of voters and candidates in real time. This project presents the design and implementation of an Election Management System developed in C++ using the Qt framework, with a focus on the practical application of fundamental Data Structures and Algorithms concepts. The system utilizes a queue implemented through a linked list to model the voter registration and voting process in a first-come, first-served manner, while candidate information is managed using a binary search tree to enable efficient insertion, searching, and deletion. Election results are dynamically generated and ranked based on vote counts using merge sort, selection sort and bubble sort algorithms. A cross-platform graphical user interface built with Qt allows administrators to manage candidates, voters, and view real-time election results. The results demonstrate that structured data handling and algorithmic techniques can significantly improve the efficiency and reliability of election management processes. Future enhancements may include persistent data storage, network-based voting, and strengthened security mechanisms.

Keywords: *Election Management System, Data Structures and Algorithms, Queue, Linked List, Binary Search Tree, Sorting Algorithms, Qt Framework*

Contents

Acknowledgements **i**

Abstract **ii**

List of Figures **v**

List of Tables **vi**

Abbreviations **vii**

1 Introduction **1**
1.1 Background 1
1.2 Objectives 1
1.3 Motivation and Significance 1

2 Related Works **3**
2.1 Review of Existing Works 3
2.1.1 Secure Online Voting System using C Programming 3
2.1.2 Election Management System (EMS) 3
2.2 Summary 4

3 Design and Implementation **5**
3.1 System Overview 5
3.2 System Requirement Specifications 5
3.2.1 Software Requirements 5
3.2.2 Hardware Requirements 6
3.3 System Design 7
3.3.1 Architectural Design 7
3.3.2 Module or Component Design 9
3.3.3 Data Design 9
3.4 Implementation Details 10
3.4.1 Software Implementation 11
3.5 Algorithms and Flowcharts 11
3.5.1 Summary of Required Content 16

4 Results and Discussion **17**
4.1 Implemented Features 17
4.2 Results and Performance Analysis 17
4.3 Comparison with Objectives or Existing Systems 18
4.4 Challenges and Limitations 18
4.5 Discussion 18

5 Conclusion and Future Works **20**
5.1 Limitations 20
5.2 Future Enhancements 20

References **22**

Appendix A **23**

List of Figures

3.1 System flow 8

5.1 Candidate Management 23
5.2 Voter Registration 23
5.3 Voting Queue Management 24
5.4 Result 24

List of Tables

3.1 Summary of Design and Implementation Content 16

Abbreviations

AGL Apple Graphics Library.

BST Binary Search Tree.

CRUD Create, Read, Update, Delete.

DSA Data Structures and Algorithms.

FIFO First-In, First-Out.

GUI Graphical User Interface.

IDE Integrated Development Environment.

OS Operating System.

SDK Software Development Kit.

UI User Interface.

Chapter 1

Introduction

1.1 Background

An election is a formal group decision-making process whereby a portion or all of a population or group votes to chooses an individual or multiple individuals to hold public office or other position of responsibility Wikipedia contributors (2026). Managing election data efficiently is critical for transparency and accuracy, but traditional manual processes are time-consuming, error-prone, and difficult to scale. This creates a need for automated systems that can handle voter registration, candidate management, vote counting, and result generation reliably.

This project presents a desktop-based Election Management System developed in C++ using the Qt framework, applying core Data Structures and Algorithms (DSA) concepts. The system uses a queue implemented via a linked list to manage voter registration and voting in a first-come, first-served manner, while candidate data is organized using a binary search tree for efficient insertion, search, listing and deletion. Sorting algorithms such as merge sort, selection sort and bubble sort are used to dynamically rank results. The Qt-based graphical interface allows administrators to efficiently manage candidates, voters, and view real-time election outcomes, demonstrating the practical application of Data Structures and Algorithms (DSA) in a real-world scenario.

1.2 Objectives

The main objectives achieved by the project are:

- To implement a voter queuing mechanism using a Queue built on a Linked List, following the First-In, First-Out (FIFO) principle.
- To manage candidate data efficiently using a Binary Search Tree (BST), supporting fast insertion, deletion, and traversal operations.
- To apply sorting algorithms (Bubble Sort, Merge Sort and Selection Sort) to rank candidates by vote count for result generation.
- To design and build a cross-platform graphical user interface using the Qt framework for interactive management of election data.

1.3 Motivation and Significance

The motivation behind selecting the Election Management System as a mini-project stems from the upcoming parliamentary elections in Nepal and the growing need for

efficient, transparent, and reliable management of electoral data. Traditional election processes are largely manual, time-consuming, and prone to errors, making it difficult to handle large numbers of voters, candidates, and ballots efficiently. While some election tasks can be managed using existing tools, most are fragmented, unstructured, or lack the ability to simulate real-time scenarios, highlighting the need for a centralized and systematic solution.

This project was undertaken to bridge the gap between theoretical data structure concepts and their practical application in a real-world, high-stakes scenario. Unlike manual processes, the proposed system provides a structured and automated approach to voter registration, candidate management, vote counting, and result ranking. It uses basic Data Structures and Algorithms (DSA), such as queues to handle voter registration, binary search trees to organize candidate information, and sorting methods to rank election results efficiently and accurately.

The implemented system combines features such as a modular architecture, cross-platform Graphical User Interface (GUI) using the Qt framework, and real-time result generation, making it a complete solution for simplified election simulation and management. Its significance lies not only in demonstrating the practical use of DSA principles but also in providing a foundation for future enhancements, including persistent data storage, multi-constituency support, and secure authentication mechanisms. Overall, the project emphasizes both academic learning and potential practical relevance, illustrating how computational techniques can improve transparency and efficiency in electoral processes.

Chapter 2

Related Works

2.1 Review of Existing Works

The field of election management system has been explored through various implementations, providing insights into automating election processes such as voter registration, candidate management, vote counting, and result generation. These existing works show how data handling techniques, user interfaces, and algorithms can be applied to enhance efficiency and transparency.

2.1.1 Secure Online Voting System using C Programming

The *Secure Online Voting System* repository Ganesh-1433 (2025) implements a basic voting platform using C programming and file handling. It features administrative and voter panels, user validation, vote casting, and result calculation, demonstrating essential functions of vote handling and election management through procedural programming and structured data storage.

Strengths:

- Secure vote casting with user validation and administrative control.
- Structured storage of election data using files, preventing duplicate votes.

Limitations:

- Limited use of advanced data structures, relying primarily on structs and arrays.
- Lacks a graphical user interface and real-time result visualization.

Comparison with Election Management System: The Election Management System applies core Data Structures (queues, linked lists, binary search trees,sorting) for efficient data management and includes a GUI using Qt, improving usability and scalability over basic file-based implementations.

2.1.2 Election Management System (EMS)

The *Election Management System* developed with Java and Swing Uttamshr10 (2025) provides a graphical user interface for administering elections. It supports basic Create, Read, Update, Delete (CRUD) operations for voters and candidates, as well as vote counting and winner declaration.

Strengths:

- Intuitive GUI that allows administrators to manage election data.

- Supports voter and candidate handling with database integration.

Limitations:

- Data structure usage and algorithmic efficiency are not emphasized.
- Limited support for advanced election features such as result sorting based on dynamic criteria.

Comparison with Election Management System: While my project includes a GUI, it focuses more on database and CRUD operations without implementing structured algorithms for efficient data handling. The Election Management System integrates core DSA concepts with a cross-platform GUI to enhance performance and user experience.

2.2 Summary

Existing election and voting system projects provide foundational methods for automating election processes, from simple C programs to Java-based GUI applications. While these works offer useful lessons in user interface design and vote processing, most do not emphasize efficient use of data structures, algorithmic sorting, or cross-platform desktop interfaces. The proposed Election Management System addresses these gaps by combining structured data handling, algorithmic result computation, and a Qt-based GUI to create a more efficient and educational election management tool.

Chapter 3

Design and Implementation

This chapter describes the design and implementation of the Election System project. It explains how the system was structured, developed, and realized in practice. The system demonstrates practical applications of Linked Lists, Queues, Binary Search Trees, and Sorting Algorithms in a real-world voting scenario.

3.1 System Overview

The Election System is a desktop application that manages the complete election process from candidate registration to result declaration. The system includes the following major components:

- **Candidate Management**: Handles candidate registration, deletion, and in-order display using a Binary Search Tree (BST).
- **Voter Registration**: Maintains a dynamic list of voters using a Singly Linked List.
- **Voting Queue Management**: Implements a First-In, First-Out (FIFO) Queue for fair voter line processing.
- **Vote Processing and Result Computation**: Processes votes, ranks candidates using Bubble, Selection, or Merge Sort, and declares winners.

Component Interaction:: All modules interact via a shared AppState object. User Interface (UI) tabs use Qt's signal-slot mechanism to update views when data changes, ensuring consistency and loose coupling between modules.

3.2 System Requirement Specifications

This section specifies the requirements necessary to develop, deploy, and execute the system.

3.2.1 Software Requirements

Development Tools and Environment::

- Programming languages used: C++
- Frameworks and Integrated Development Environment (IDE) : QT Creator
- Version Control: Git
- Operating Systems: Windows 10+, macOS 11+, Linux (Ubuntu 20.04+)

Functional Requirements

- Candidate Registration with Binary Search Tree (BST) insert/search/delete operations
- Voter registration using the linked list
- Queue Management using FIFO operations
- Vote Processing: Validate voters and update candidate vote counts
- Result Computation using selectable sorting algorithms
- Winner Declaration based on maximum votes

Non-Functional Requirements

- Usability: Intuitive tab-based graphical interface for easy interaction.
- Performance: Efficient operations—BST search in O(log n), Queue enqueue/dequeue in O(1).
- Reliability: Graceful handling of empty structures, duplicate IDs, and invalid inputs.
- Maintainability: Clear separation of DSA logic and UI components.
- Portability: Cross-platform support on Windows, macOS, and Linux via Qt.

3.2.2 Hardware Requirements

Minimum System Configuration:

- Processor: Intel Core i3 / AMD 2.0 GHz+ or equivalent
- RAM: 4 GB
- Display: 1280×720 resolution
- Graphics: Integrated graphics with OpenGL 2.1 support
- Supported Operating System (OS): Windows 10+, macOS 11+, Linux (Ubuntu 20.04+

3.3 System Design

3.3.1 Architectural Design

The Election Management System follows a layered architectural design with a clear separation between data handling, application logic, and user interface components. This design approach enhances modularity, maintainability, and clarity of the overall system.

The architecture is divided into three primary layers:

1. **Data Structure Layer**
   This layer consists of pure C++ implementations of core Data Structures and Algorithms. It includes singly linked lists for voter registration, queues for maintaining the voting order, binary search trees for storing and retrieving candidate information, and sorting algorithms for ranking election results. The layer has no dependency on the graphical user interface, ensuring portability and reusability of the algorithmic logic.

2. **Application State Layer**
   The application state layer coordinates interactions among all data structures. A centralized AppState structure stores and manages election-related data such as registered voters, candidates, and the voting queue. This shared state guarantees data consistency and synchronized access across all system modules.

3. **User Interface Layer**
   The user interface layer is developed using the Qt framework and provides a tab-based graphical environment for system interaction. Each tab handles specific functionalities including candidate management, voter registration, voting queue operations and result visualization. Qt's signal-slot mechanism is used for communication between components, enabling loose coupling and real-time interface updates.

This layered architecture supports independent development of system components, simplifies future enhancements, and effectively demonstrates the practical application of data structures and algorithms within a realistic election management environment.

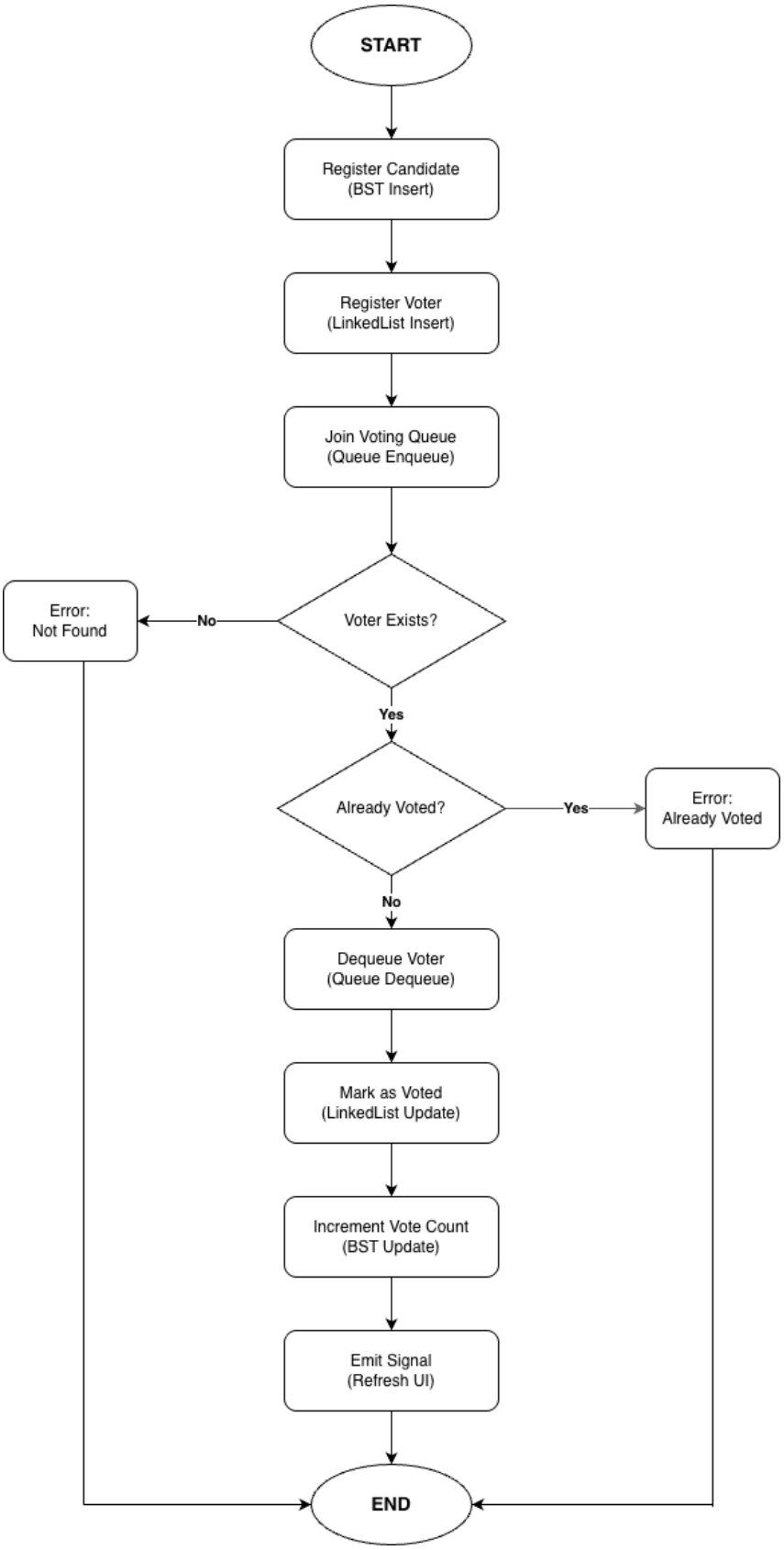

Figure 3.1: System flow

3.3.2 Module or Component Design

The major modules of the system are:

- **Main Window Module:** Controls the whole application. It creates all tabs, connects modules, and manages the app flow.
- **Candidate Module:** Handles candidate registration using a Binary Search Tree (BST). It allows insert, search, and delete operations and shows candidates in sorted order.
- **Voter Module:** Manages voter records using a linked list. It supports adding voters, searching by ID, deleting voters, and tracking voting status.
- **Voting Queue Module:** Manages the voting line using a FIFO queue. It adds voters to the queue, processes them in order, and checks eligibility before voting.
- **Results Module:** Calculates election results. It gets candidates from the BST, sorts them using the selected algorithm (Bubble, Selection, or Merge), and declares the winner.
- **Application State Module:** Stores shared data for the whole system. It keeps single instances of all data structures and allows all modules to access them safely.

Each module works independently while communicating through well-defined Qt signals and slots, ensuring easy maintenance and allowing new features to be added without affecting other components.

3.3.3 Data Design

The Election System does not use an external database. Instead, all data is stored in in-memory data structures designed for efficient algorithm performance. The data design focuses on how information is organized within each DSA component.

Data Structure Overview

The system uses pointer-based structures. Voters are stored in a linked list, the voting line is managed using a queue, and candidates are maintained in a Binary Search Tree (BST). Sorting of results is handled using a separate result structure.

Voter Node Structure (Linked List)

Stores voter information and voting status. Each node points to the next voter.

```
struct VoterNode {
    int voterId;          // Primary key (unique)
    QString name;         // Voter full name
    bool hasVoted;        // Voting status flag
    int votedForId;       // Foreign key to candidate (-1 if not vot
    VoterNode* next;      // Pointer to next voter in list
};
```

Queue Node Structure (Voting Queue)

Represents voters waiting in the voting line using FIFO order.

```
struct QueueNode {
    int voterId;          // References voter in linked list
    QueueNode* next;      // Pointer to next voter in queue
};
```

BST Node Structure (Candidates)

Stores candidate details in a Binary Search Tree for efficient search and sorted traversal.

```
struct BSTNode {
    int candidateId;      // Primary key (BST search key)
    QString name;         // Candidate name
    QString party;        // Party affiliation
    int voteCount;        // Accumulated votes
    BSTNode* left;        // Left subtree (smaller IDs)
    BSTNode* right;       // Right subtree (larger IDs)
};
```

Candidate Result Structure (Sorting)

Used during result computation and sorting based on vote count.

```
struct CandidateResult {
    int id;               // Candidate ID
    QString name;         // Candidate name
    QString party;        // Party name
    int votes;            // Vote count (sort key)
};
```

3.4 Implementation Details

This section describes how the system design was translated into an actual working system.

3.4.1 Software Implementation

The Election Management System was implemented using C++ and the Qt framework to provide a responsive and user-friendly desktop application. Qt Widgets were used to design the graphical user interface, which includes separate tabs for candidate management, voter registration, voting operations, result display, and data structure visualization.

The core logic of the system is based on fundamental Data Structures and Algorithms. Linked Lists are used for managing voter records, Queues are used to maintain the voting order, Binary Search Trees store and retrieve candidate information efficiently, and Sorting algorithms are applied to rank election results. All data structures were implemented as pure C++ classes without any Qt dependency, ensuring clean separation between logic and interface.

Qt's signal-slot mechanism was used to connect user actions such as button clicks to backend logic. A shared application state maintains consistency across all modules, allowing updates in one tab to be reflected automatically in others. The system also includes real-time visualization of data structures to support better understanding of DSA concepts.

3.5 Algorithms and Flowcharts

This section describes the key algorithms and workflows that govern the operation of the Election system along with their pseudo code and complexity.

Algorithm 1: Linked List Insert at End

```
ALGORITHM insertVoter(id, name):
    newNode = createNode(id, name)
    IF head = NULL THEN
        head = newNode
        RETURN
    END IF
    current = head
    WHILE current.next != NULL DO
        current = current.next
    END WHILE
    current.next = newNode
END ALGORITHM
```

Time Complexity: O(n)
Space Complexity: O(1)

Algorithm 2: Queue Enqueue and Dequeue

```
ALGORITHM enqueue(voterId):
    newNode = createNode(voterId)
```

```
    IF rear = NULL THEN
        front = rear = newNode
        RETURN
    END IF
    rear.next = newNode
    rear = newNode
END ALGORITHM

ALGORITHM dequeue():
    IF front = NULL THEN
        PRINT "Queue Underflow"
        RETURN
    END IF

    temp = front
    front = front.next

    IF front = NULL THEN
        rear = NULL
    END IF

    free(temp)
END ALGORITHM
```

Time Complexity: O(1)
Space Complexity: O(1)

Algorithm 3: BST Insert, Search and Delete

```
ALGORITHM insertCandidate(node, id, name, party):
    IF node = NULL THEN
        RETURN createNode(id, name, party)
    END IF
    IF id < node.candidateId THEN
        node.left = insertCandidate(node.left, id, name, party)
    ELSE IF id > node.candidateId THEN
        node.right = insertCandidate(node.right, id, name, party)
    END IF
    RETURN node
END ALGORITHM

ALGORITHM searchCandidate(node, id):
    IF node = NULL OR node.candidateId = id THEN
        RETURN node
    END IF
```

```
    IF id < node.candidateId THEN
        RETURN searchCandidate(node.left, id)
    ELSE
        RETURN searchCandidate(node.right, id)
    END IF
END ALGORITHM

ALGORITHM deleteCandidate(node, id):
    IF node = NULL THEN
        RETURN NULL
    END IF

    IF id < node.candidateId THEN
        node.left = deleteCandidate(node.left, id)
    ELSE IF id > node.candidateId THEN
        node.right = deleteCandidate(node.right, id)
    ELSE
        IF node.left = NULL THEN
            temp = node.right
            free(node)
            RETURN temp
        ELSE IF node.right = NULL THEN
            temp = node.left
            free(node)
            RETURN temp
        END IF

        temp = findMin(node.right)
        node.candidateId = temp.candidateId
        node.name = temp.name
        node.party = temp.party
        node.right = deleteCandidate(node.right, temp.candidateId)
    END IF

    RETURN node
END ALGORITHM
```

Time Complexity: Average Case: O(log n), Worst Case: O(n)
Space Complexity: O(log n)

Algorithm 4: Bubble Sort

```
ALGORITHM bubbleSort(arr, n):
    FOR i = 0 TO n-2 DO
        FOR j = 0 TO n-i-2 DO
            IF arr[j].votes < arr[j+1].votes THEN
```

```
                swap(arr[j], arr[j+1])
            END IF
        END FOR
    END FOR
END ALGORITHM
```

Time Complexity: $O(n^2)$
Space Complexity: O(1)

Algorithm 5: Selection Sort

```
ALGORITHM selectionSort(arr, n):
    FOR i = 0 TO n-2 DO
        maxIndex = i
        FOR j = i+1 TO n-1 DO
            IF arr[j].votes > arr[maxIndex].votes THEN
                maxIndex = j
            END IF
        END FOR
        swap(arr[i], arr[maxIndex])
    END FOR
END ALGORITHM
```

Time Complexity: $O(n^2)$
Space Complexity: O(1)

Algorithm 6: Merge Sort

```
ALGORITHM mergeSort(arr, left, right):
    IF left >= right THEN RETURN
    mid = (left + right) / 2
    mergeSort(arr, left, mid)
    mergeSort(arr, mid+1, right)
    merge(arr, left, mid, right)
END ALGORITHM

ALGORITHM merge(arr, left, mid, right):
    temp = empty array
    i = left, j = mid+1
    WHILE i <= mid AND j <= right DO
        IF arr[i].votes >= arr[j].votes THEN
            temp.append(arr[i])
            i = i + 1
        ELSE
            temp.append(arr[j])
            j = j + 1
```

```
        END IF
    END WHILE
    WHILE i <= mid DO
        temp.append(arr[i])
        i = i + 1
    END WHILE
    WHILE j <= right DO
        temp.append(arr[j])
        j = j + 1
    END WHILE
    Copy temp to arr[left..right]
END ALGORITHM
```

Time Complexity: O(n log n)
Space Complexity: O(n)

3.5.1 Summary of Required Content

Table 3.1: Summary of Design and Implementation Content

Section / Component	**Software Project (Election System)**
System Overview	Desktop application managing the complete voting process using core DSA structures: Linked List, Queue, Binary Search Tree (BST), and Sorting algorithms. Provides a tab-based UI with candidate management, voter registration, voting queue and results computation.
Software Requirements	C++, Qt Creator, Git
System Architecture	Layered architecture: Data Structure Layer (pure C++ DSA classes), Application State Layer (AppState shared across modules), and UI Layer (Qt tabs with signal-slot communication).
Module / Component Design	Candidate management (BST), Voter registration (Linked List), Voting Queue management (FIFO Queue), Results computation (Sorting algorithms: Bubble, Selection, Merge).
Data Design	In-memory data structures: VoterNode (Linked List), QueueNode (Queue), BSTNode (Candidates), CandidateResult (Sorting). No external database.
Implementation Details	Pointer-based DSA structures, header-only C++ classes, Qt signal-slot mechanism for cross-tab synchronization, tab-based UI integration.
Algorithms and Flowcharts	Linked List insertion, Queue enqueue/dequeue, BST insertion/search, Bubble/Selection/Merge Sort for results computation.

Chapter 4

Results and Discussion

4.1 Implemented Features

The following key features were successfully implemented:

- **Candidate Management:** Candidates can be registered, searched, and deleted using a Binary Search Tree (BST). This ensures efficient lookup and maintains candidates in sorted order automatically.
- **Voter Registration:** Voters are registered using a singly linked list. Each voter record stores ID, name, voting status, and candidate selection. Searching, deletion, and status tracking are supported.
- **Voting Queue Management:** The system manages the voting line in a first-come-first-serve manner using a queue. Voters are enqueued and dequeued efficiently, ensuring fairness and correct voting order.
- **Winner Declaration:** Election results are computed by retrieving candidates from the BST and sorting them based on votes using selectable algorithms: Bubble Sort, Selection Sort, or Merge Sort.
- **Tab-Based Graphical Interface:** A Qt-based interface provides separate tabs for candidate management, voter registration, voting operations, and results visualization. Real-time updates are handled using the Qt signal-slot mechanism.

4.2 Results and Performance Analysis

The system operated correctly across all modules. Candidate operations through the BST were fast and reliable, while linked list management ensured accurate voter records. The voting queue maintained proper order and prevented duplicate voting, safeguarding election integrity.

Performance observations indicate that BST searches are efficient, providing near-instant retrieval of candidate data in average scenarios. Linked list insertion is adequate for moderate numbers of voters, and queue operations remain constant in time complexity. Bubble and Selection Sort were slower for larger candidate sets, whereas Merge Sort proved to be the most efficient and scalable method for ranking results.

Memory usage is dynamic due to pointer-based structures, allowing flexible management of voter and candidate information. The Qt interface remained responsive, even during intensive operations like vote counting and result computation, demonstrating effective real-time synchronization across tabs.

4.3 Comparison with Objectives or Existing Systems

The Election Management System successfully meets its main objectives by implementing core data structures—Linked List, Queue, BST, and Sorting algorithms—with correct performance and behavior. Candidate and voter management, vote processing, and result computation are efficiently handled, while the tab-based Qt interface provides an intuitive, real-time interactive experience. The system also visualizes data structures, helping users understand how votes and candidates are organized and processed.

Compared to existing traditional systems discussed in Chapter 2, which focus on basic CRUD operations, file storage, or GUI without emphasizing algorithmic efficiency, this system integrates structured data handling and algorithmic sorting to optimize performance. It ensures faster candidate retrieval, accurate vote counting, and dynamic result computation, offering an educational yet practical platform that combines functionality, efficiency, and usability in a cross-platform desktop environment. The obtained results align closely with the original project objectives. The system successfully automated the process of question paper generation, reducing manual effort while maintaining structure, relevance, and topic diversity. The integration of machine learning enabled intelligent question selection rather than static or rule-based generation.

4.4 Challenges and Limitations

Managing memory with pointers in linked lists, queues, and BSTs required careful attention to avoid leaks, while keeping data synchronized across tabs was handled using Qt's signal-slot mechanism. Sorting large candidate sets efficiently necessitated Merge Sort, and input validation prevented duplicate or invalid entries. On macOS, linker errors due to deprecated AGL references were resolved by patching Qt spec files, using the `sdk_no_version_check` flag, and automating the process with shell scripts.

Remaining limitations include lack of persistent storage, limited scalability for very large voter or candidate datasets, minimal security, single-machine operation, and potential BST imbalance in worst-case scenarios.

4.5 Discussion

The system effectively demonstrates the practical application of fundamental data structures and algorithms in a voting context. Queue operations achieved constant-time processing, BST operations provided efficient average-case performance, and sorting algorithms correctly ranked candidates.

The tab-based interface allows intuitive interaction and real-time feedback, while dynamic visualization of voters, candidates, and the voting queue enhances understanding of underlying data structures. Although primarily designed for educational purposes, the system confirms that careful selection of algorithms, modular design,

and layered architecture result in a functional, maintainable, and responsive application. Future enhancements could include persistent storage, advanced security, multi-user support, and balanced BST implementations for optimized performance.

Chapter 5

Conclusion and Future Works

The Election Management System was successfully designed and implemented to provide a functional and educational platform for managing elections. The system effectively demonstrates the application of fundamental data structures—Linked Lists for voter management, Queues for maintaining voting order, and BST for candidate handling—while integrating sorting algorithms such as Bubble Sort, Selection Sort, and Merge Sort for result computation. Merge Sort, in particular, proved efficient for handling larger candidate datasets.

The Qt-based tabbed interface enables seamless communication between modules using signals and slots, ensuring real-time synchronization of voter, candidate, and queue data. The modular and visual approach enhances both usability and understanding of core DSA concepts. Overall, the system meets its primary objectives by combining practical data structure implementation with an intuitive desktop environment, bridging the gap between theory and application.

5.1 Limitations

Despite its successful implementation, the system has several limitations:

- Data is not stored persistently; all voter and candidate records are lost when the application closes.
- Scalability is limited for very large numbers of voters or candidates.
- Security features are minimal, with no authentication or encryption.
- Multi-user and network support is not available.
- On macOS, initial compatibility issues arose due to deprecated Apple Graphics Library (AGL) references and mismatched Qt Software Development Kit (SDK) versions, which required patching and automation scripts.

These limitations are primarily due to the educational scope and resource constraints of the project.

5.2 Future Enhancements

Future improvements could expand functionality, scalability, and usability:

- **Persistent Storage:** Integrate a database like SQLite or Mysql or postgres for permanent record-keeping.
- **Enhanced Security:** Add authentication, role-based access, and encryption to safeguard data.

- **Multi-User Support:** Enable concurrent access across a network for distributed election management.
- **Advanced Analytics:** Include additional sorting and ranking methods, statistical vote analysis, and graphical result visualization.
- **User Interface Improvements:** Modernize UI components and improve accessibility.

These improvements feel like the next step and would make the system more effective and supportive as an intelligent election management system suitable for real-world applications.

References

Ganesh-1433 (2025). Secure online voting system using c programming.

Uttamshr10 (2025). Election management system using java swing.

Wikipedia contributors (2026). Election.

Appendix A

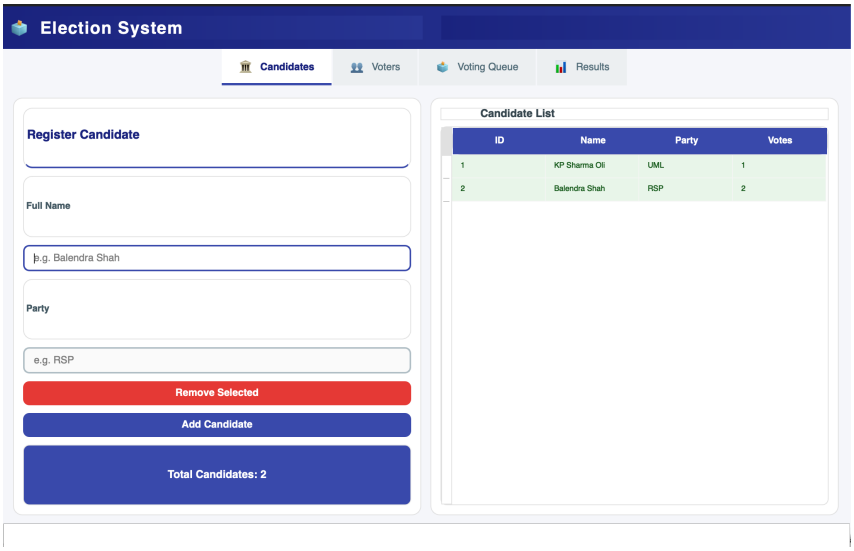

Figure 5.1: Candidate Management

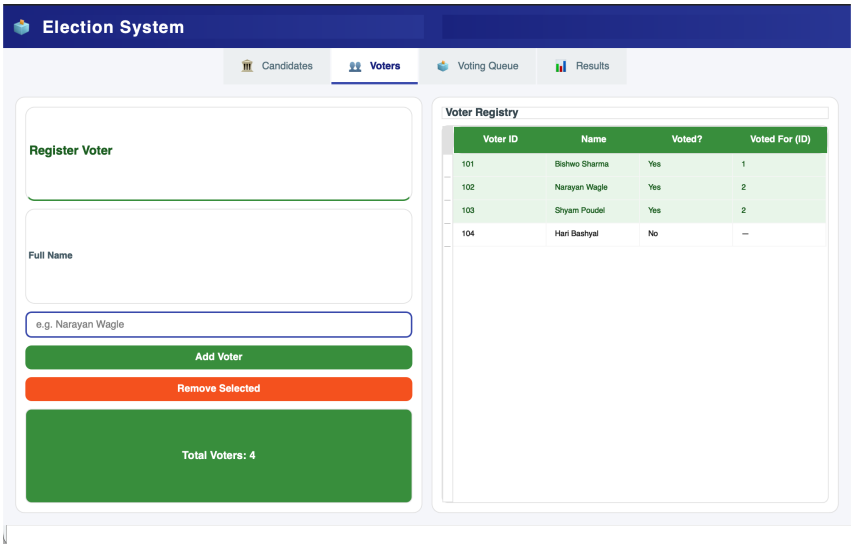

Figure 5.2: Voter Registration

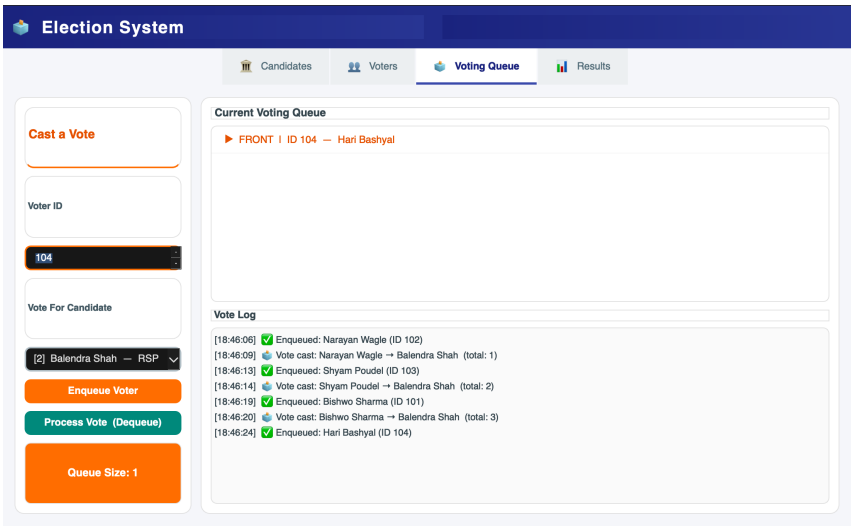

Figure 5.3: Voting Queue Management

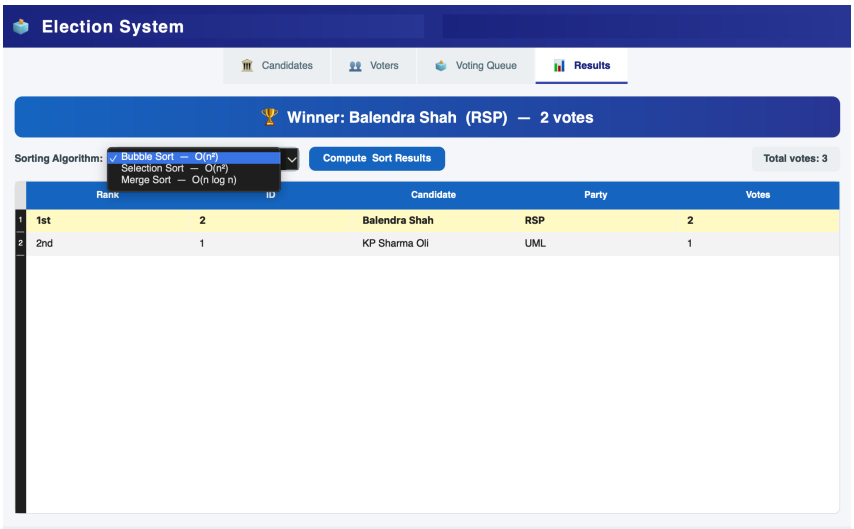

Figure 5.4: Result